

NAME : SHAH FAISAL

ROLL NO : 23I-0058

SECTION : BSE – 4A

OS LAB 04

BSSE SPRING 2026

LAB 04

Tasks:

I. Write a program that creates two child processes using `fork()`. The program should use a global variable to demonstrate how changes in the global variable are reflected in the child processes.

Each child process should increment the global variable by a specific amount and print its result.

The parent process should print the final value of the global variable after both child processes

have completed. You are not allowed to use `wait()`. Output is given below: [Estimated Time: 20 mins]

```
#include <stdio.h>
#include <unistd.h>
int rollnum=58;
int main() {
    pid_t pid1, pid2;
    pid1=fork();
    if(pid1==0) {
        rollnum+=10;
        printf("Child 1 value=%d\n",rollnum);
        return 0;
    }
    pid2=fork();
    if(pid2==0) {
        rollnum+=20;
        printf("Child 2 value = %d\n",rollnum);
        return 0;
    }
    sleep(1);
    printf("Parent: Final global variable = %d\n",rollnum);
    return 0;
}
```

```

main.c | i230058_q2.c | i230058_q3.c
1 #include <stdio.h>
2 #include <unistd.h>
3 int rollnum=58;
4 int main() {
5     pid_t pid1, pid2;
6     pid1=fork();
7     if(pid1==0) {
8         rollnum+=10;
9         printf("Child 1 value=%d\n",rollnum);
10        return 0;
11    }
12    pid2=fork();
13    if(pid2==0) {
14        rollnum+=20;
15        printf("Child 2 value = %d\n",rollnum);
16        return 0;
17    }
18    sleep(1);
19    printf("Parent: Final global variable = %d\n",rollnum);
20    return 0;
21 }
22

input
Child 1 value = 68
Child 2 value = 78
Parent: Final global variable = 58

...Program finished with exit code 0
Press ENTER to exit console.
  
```

II. Write a program that creates a single child process and then creates another child process from the first child. Each child process should print its PID and PPID. The parent process should print the PIDs of all active processes (including itself and its children) after both children have been created. You are not allowed to use wait(). Parent process details should be printed after child process runs. Output is given below: [Estimated Time: 25 min]

```

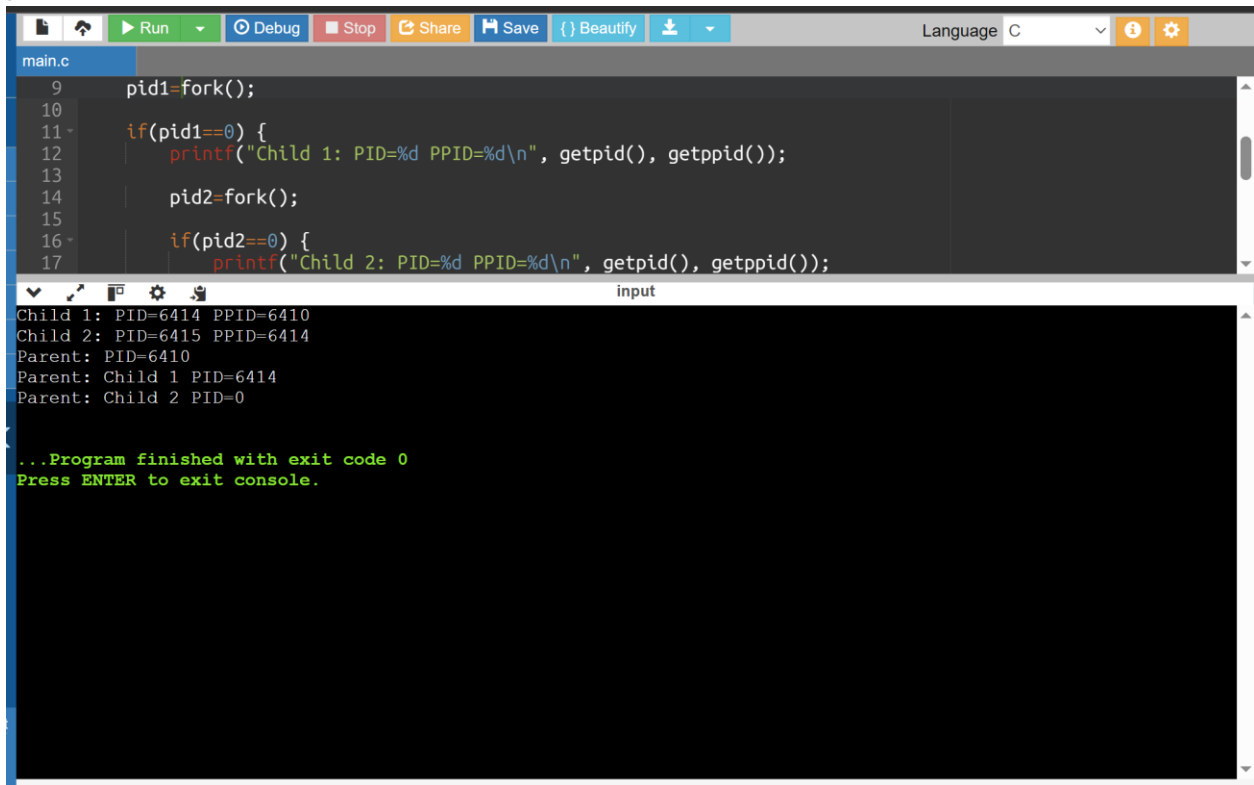
#include <stdio.h>
#include <unistd.h>
  
```

```
int rollnum=58;
```

```
int main() {
    pid_t pid1, pid2;
```

```
    pid1=fork();
```

```
if(pid1==0) {  
    printf("Child 1: PID=%d PPID=%d\n", getpid(), getppid());  
  
    pid2=fork();  
  
    if(pid2==0) {  
        printf("Child 2: PID=%d PPID=%d\n", getpid(), getppid());  
        return 0;  
    }  
  
    sleep(1);  
    return 0;  
}  
  
sleep(2);  
printf("Parent: PID=%d\n", getpid());  
printf("Parent: Child 1 PID=%d\n", pid1);  
printf("Parent: Child 2 PID=%d\n", 0);  
  
return 0;  
}
```



```
main.c  
9 pid1=fork();  
10  
11 if(pid1==0) {  
12     printf("Child 1: PID=%d PPID=%d\n", getpid(), getppid());  
13  
14     pid2=fork();  
15  
16     if(pid2==0) {  
17         printf("Child 2: PID=%d PPID=%d\n", getpid(), getppid());  
18     }  
19  
20     sleep(1);  
21     return 0;  
22 }  
23  
24 sleep(2);  
25 printf("Parent: PID=%d\n", getpid());  
26 printf("Parent: Child 1 PID=%d\n", pid1);  
27 printf("Parent: Child 2 PID=%d\n", 0);  
28  
29 return 0;  
30 }
```

```
Child 1: PID=6414 PPID=6410  
Child 2: PID=6415 PPID=6414  
Parent: PID=6410  
Parent: Child 1 PID=6414  
Parent: Child 2 PID=0  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

3. Write a program that creates three child processes sequentially. Each child process should print its PID, perform a simple task (like a loop or arithmetic operation), and then exit. The parent process should wait for each child to complete before creating the next one. After all child processes have finished, the parent should print a final message indicating that all children have terminated. You are allowed to use `fork()` statement only once. It means in whole program there should be only one `fork()` system call but have three children. Output is given below: [Estimated Time: 30 mins]

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

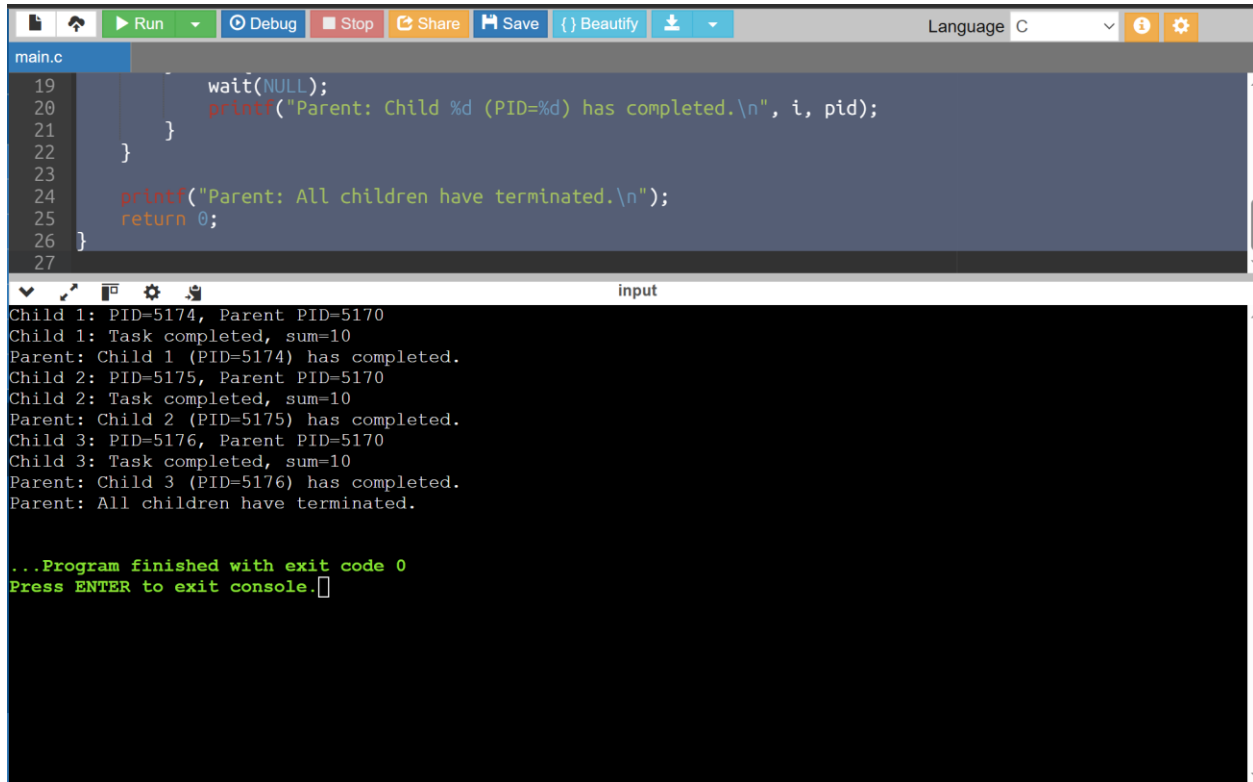
int rollnum=0;

int main() {
    pid_t pid;

    for(int i=1; i<=3; i++) {
        pid = fork();

        if(pid==0) {
            rollnum += 10;
            printf("Child %d: PID=%d, Parent PID=%d\n", i, getpid(), getppid());
            printf("Child %d: Task completed, sum=%d\n", i, rollnum);
            return 0;
        } else {
            wait(NULL);
            printf("Parent: Child %d (PID=%d) has completed.\n", i, pid);
        }
    }

    printf("Parent: All children have terminated.\n");
    return 0;
}
```



The image shows a code editor window with a file named 'main.c' and a terminal window below it. The code in 'main.c' is a C program that creates three child processes, each performing a task (summing 1 to 10) and reporting completion to the parent. The parent process waits for all children to finish before printing a final message and returning 0. The terminal output shows the execution of this program, with each child reporting its PID, parent PID, and task completion. The program finishes with exit code 0.

```
19     wait(NULL);
20     printf("Parent: Child %d (PID=%d) has completed.\n", i, pid);
21 }
22 }
23
24 printf("Parent: All children have terminated.\n");
25 return 0;
26 }
27
```

input

```
Child 1: PID=5174, Parent PID=5170
Child 1: Task completed, sum=10
Parent: Child 1 (PID=5174) has completed.
Child 2: PID=5175, Parent PID=5170
Child 2: Task completed, sum=10
Parent: Child 2 (PID=5175) has completed.
Child 3: PID=5176, Parent PID=5170
Child 3: Task completed, sum=10
Parent: Child 3 (PID=5176) has completed.
Parent: All children have terminated.

...Program finished with exit code 0
Press ENTER to exit console.
```